

Programmation Mobile

API REST et Android

2025 - 2026
Dr. Abdelkader Ouared



Agenda

Introduction

API

Fondamentaux
HTTP

API REST

Bibliothèques
HTTP

Exercice

Agenda

Introduction

API

Fondamentaux
HTTP

API REST

Bibliothèques
HTTP

Exercice

Introduction



Quand nous commençons en tant que développeur Android



Views et Layouts



Activités et Fragments



Gérer les événements

Qu'est-ce qui rend votre application professionnelle ?



Authentification



Requêtes
asynchrones

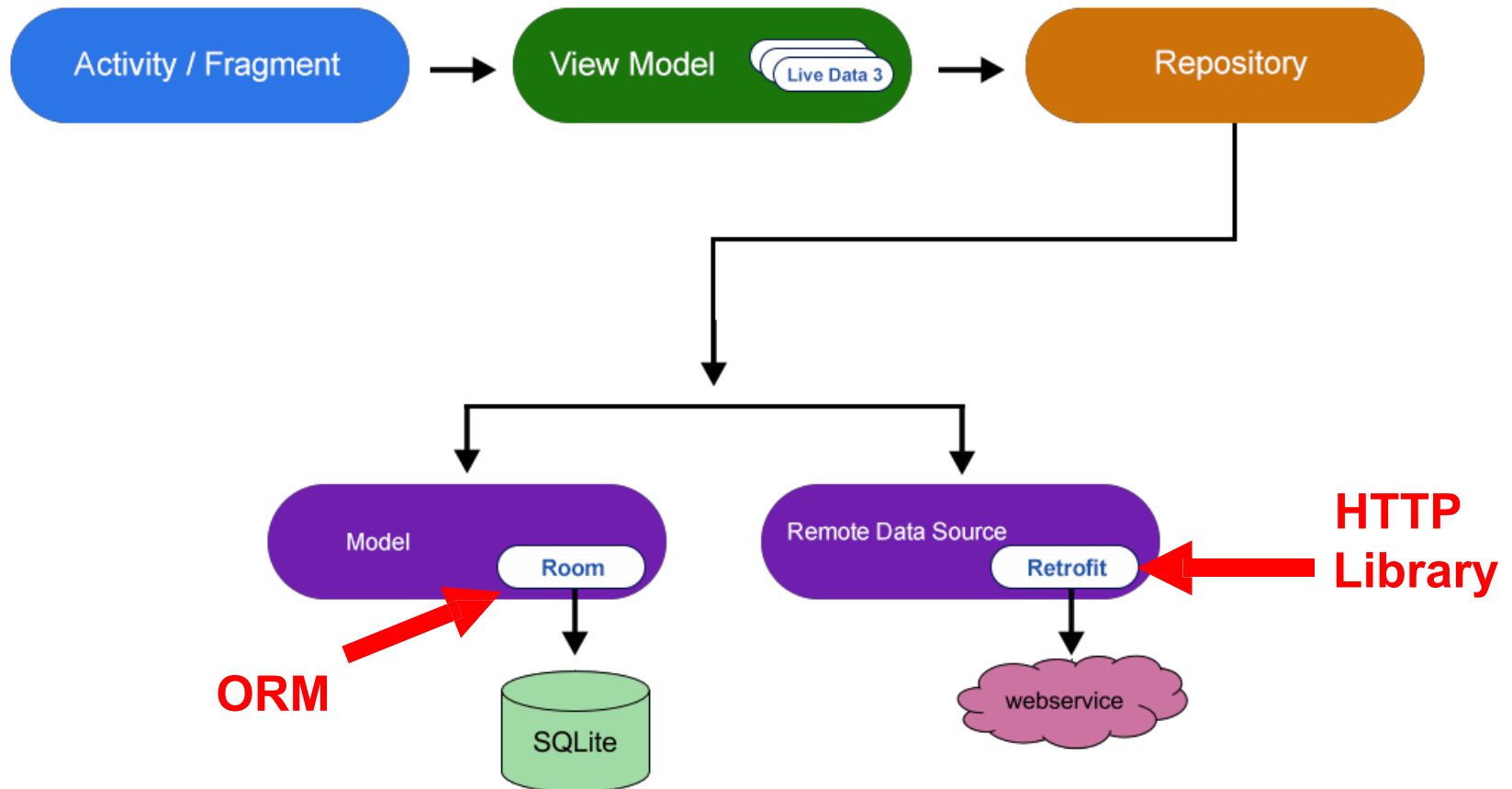


Mapper un JSON
en objets utilisables



Charger l'image,
invoquer l'API.

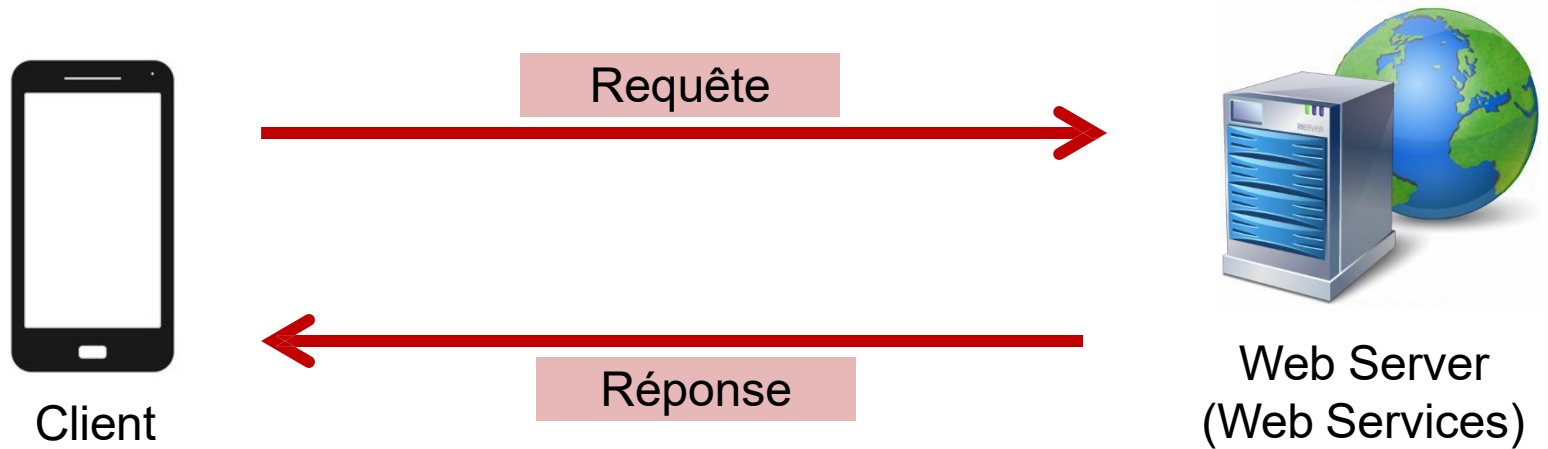
BD locale et BD distante



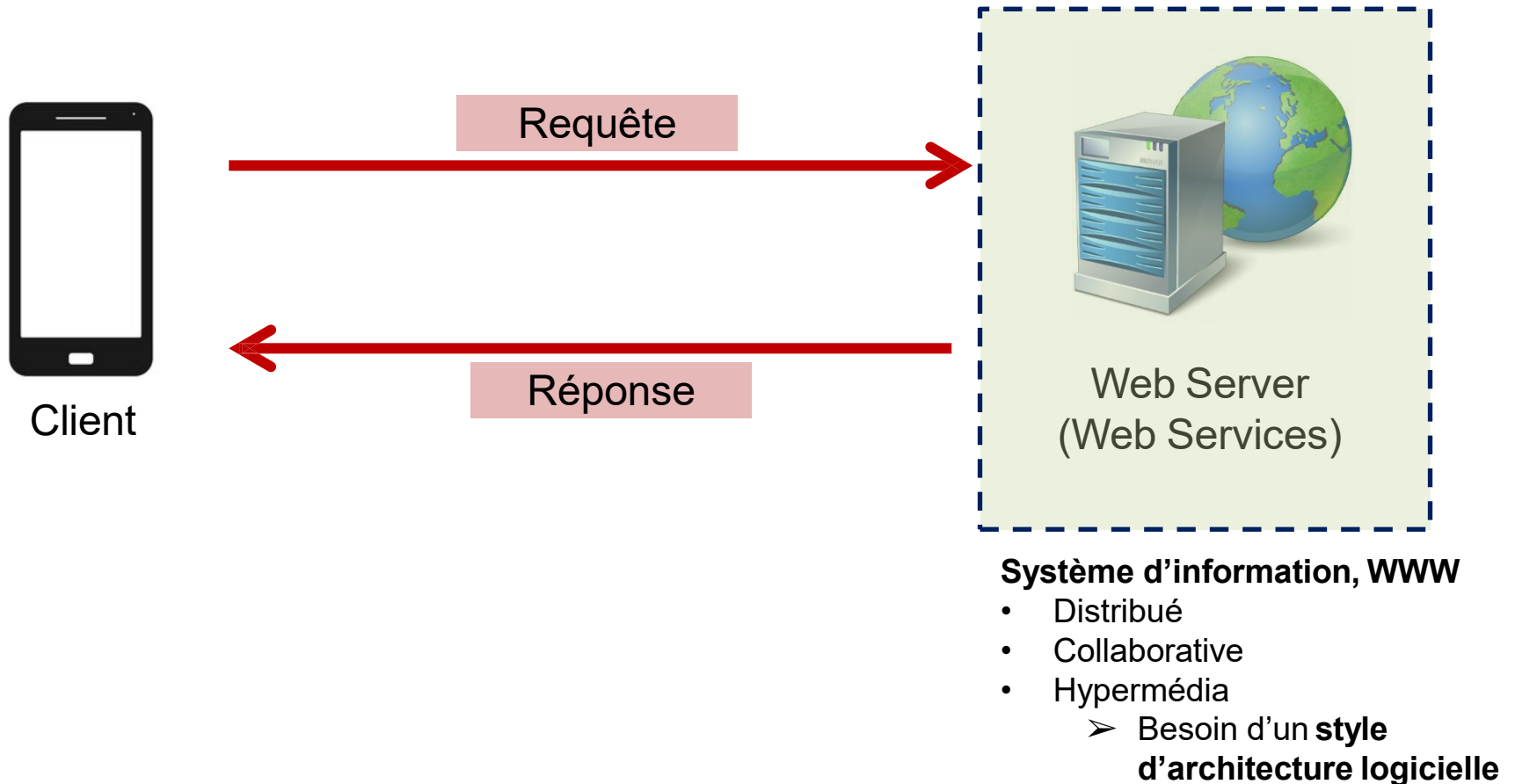
Comment exploiter les ressources trouvées sur le Web ?



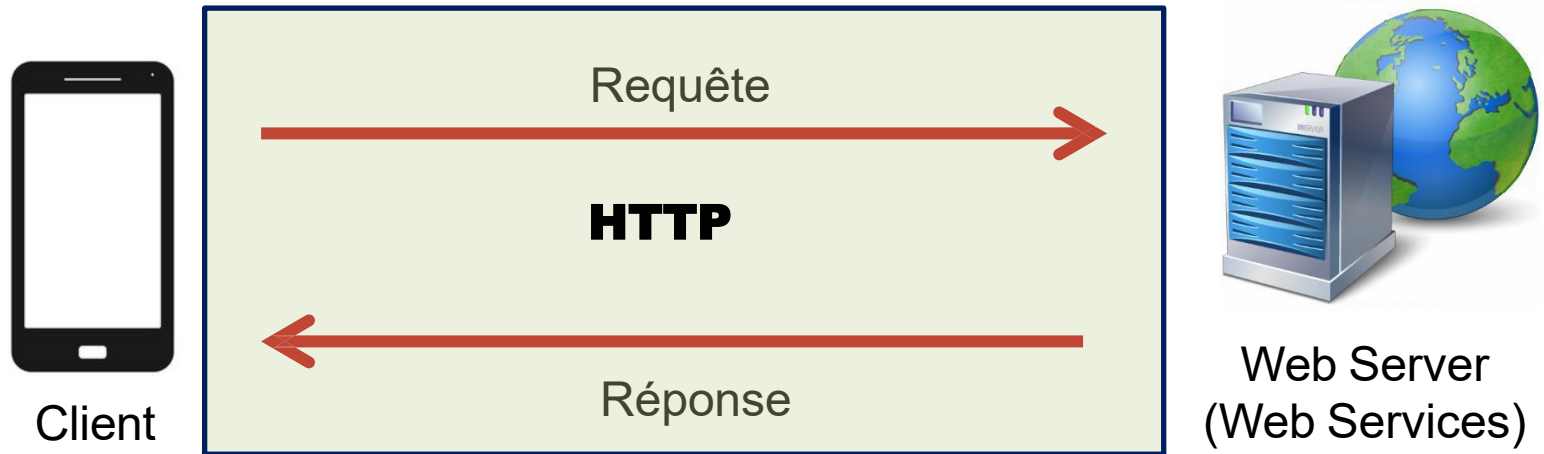
Exploiter les ressources dans le web



Caractéristiques du système d'information, WWW



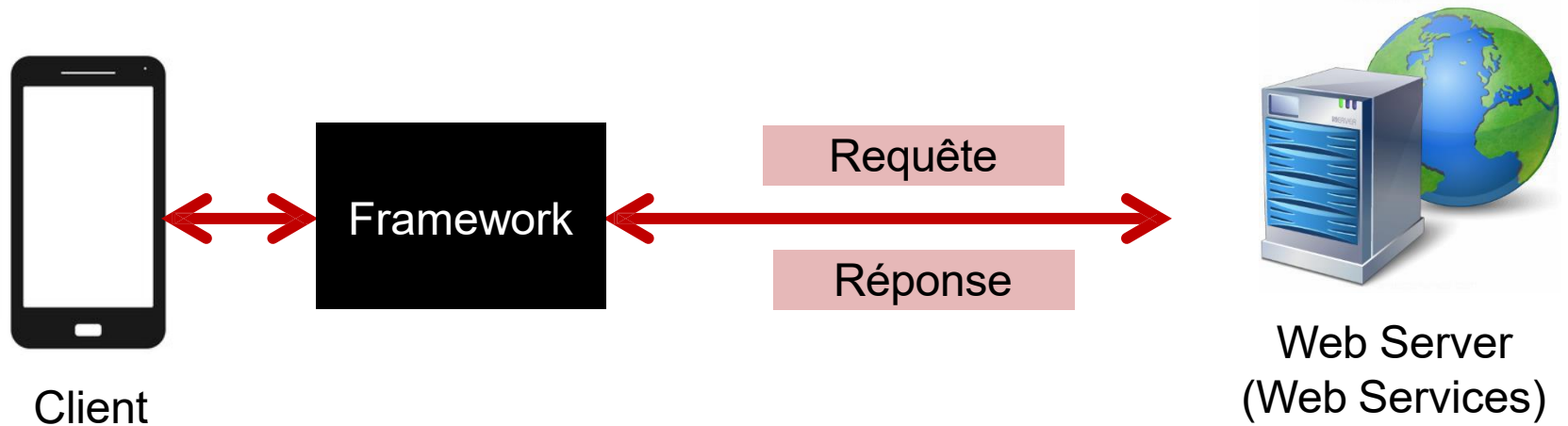
Exigences de protocole de communication



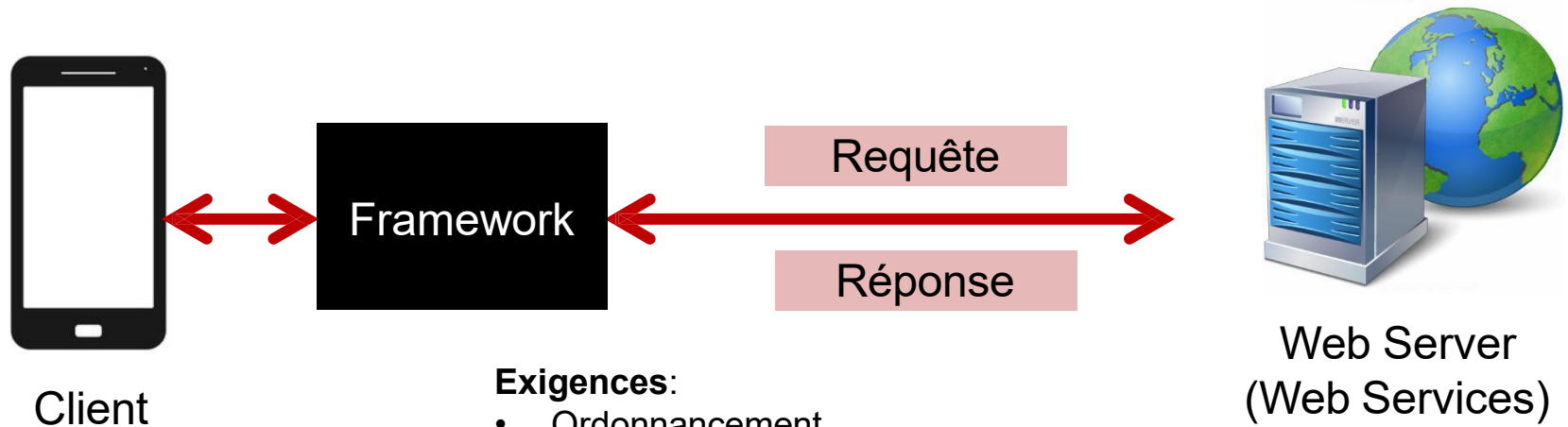
Exigences:

- Ordonnancement
- Priorisation
- Accès Multiple et concurrent
- Analyse de contenu
- Faites-le de manière asynchrone
- Avertir le composant si le travail se termine
 - Développement à partir de zéro ☹
 - Faciliter la vie de programmeur

Utiliser un framework au niveau de client



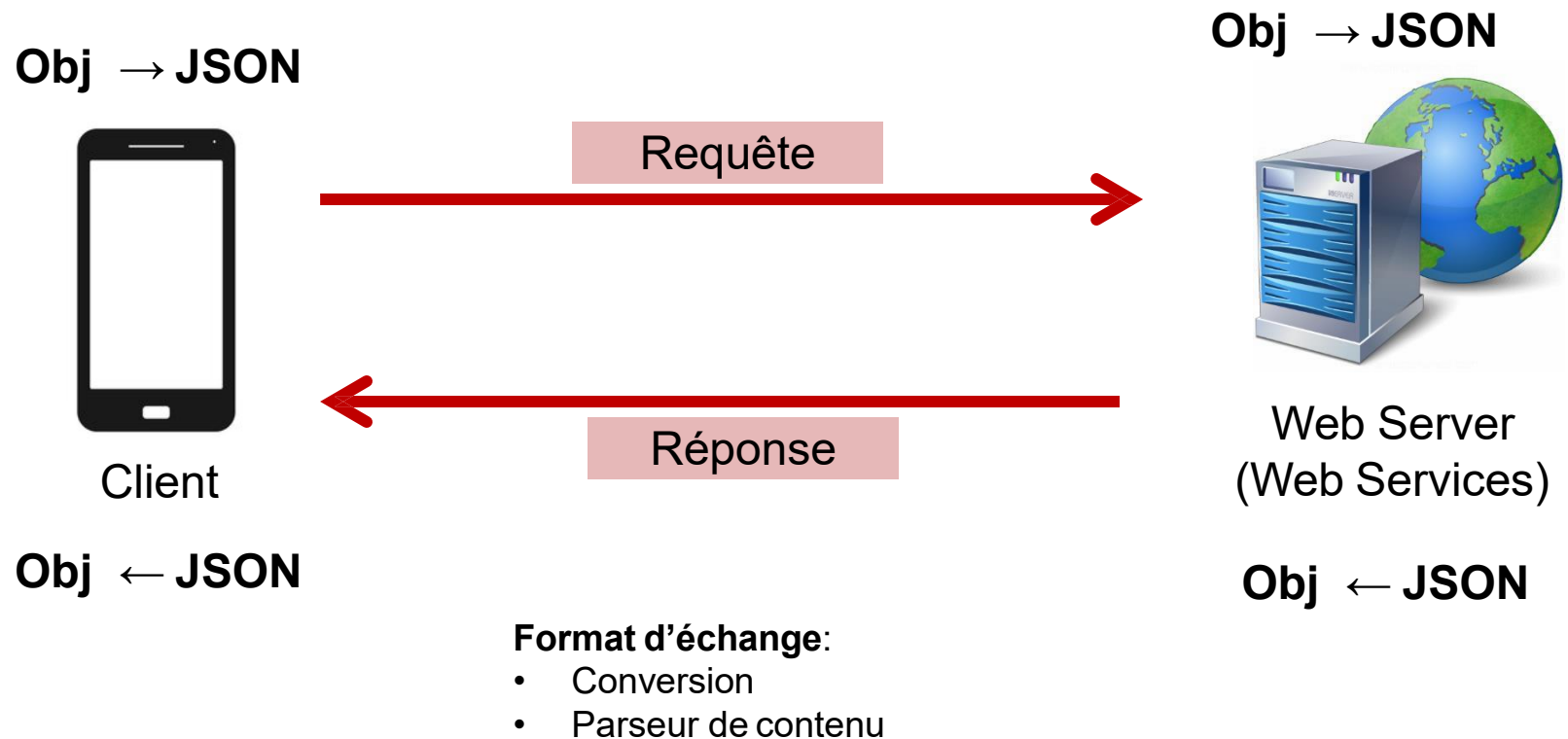
Exploiter les ressources dans le web



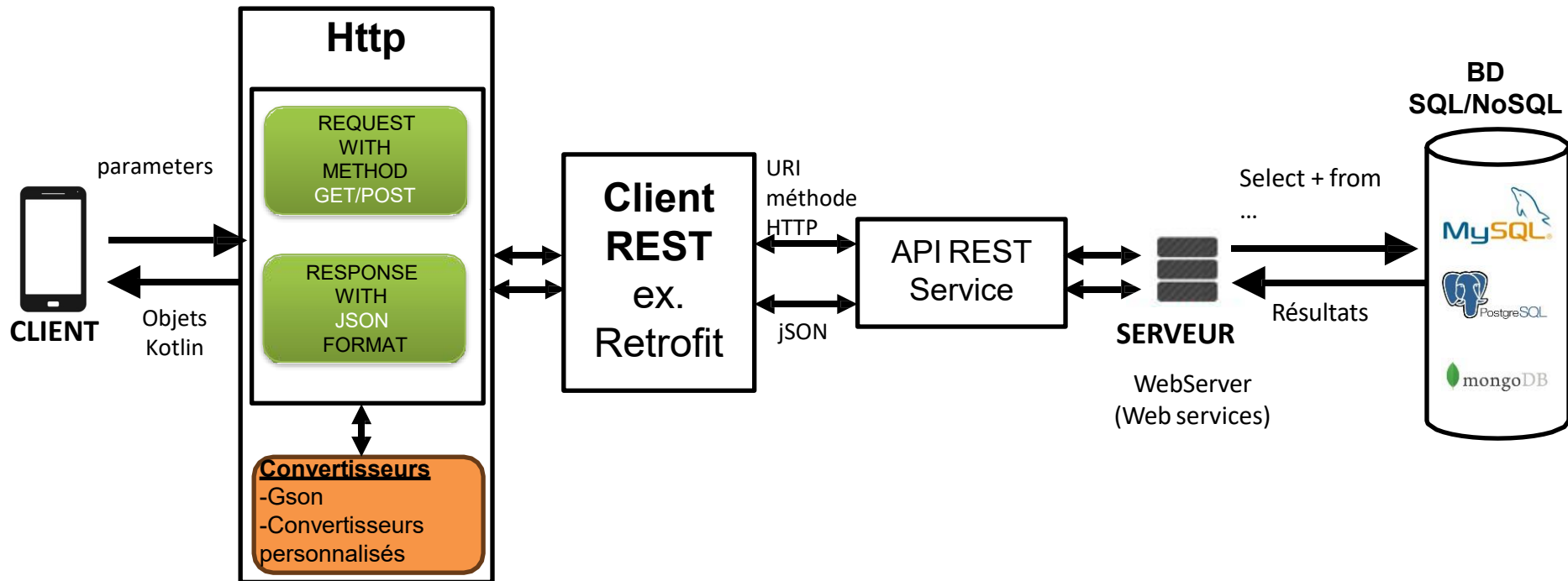
Exigences:

- Ordonnancement
- Priorisation
- Accès Multiple et concurrent
- Analyse de contenu
- Faites-le de manière asynchrone
- Avertir le composant si le travail se termine
 - Développement à partir de zéro ☹️
 - Faciliter la vie de programmeur

Besoin de format d'échange standard



Cas de Android studio



Agenda

Introduction

API

Fondamentaux
HTTP

API REST

Bibliothèques
HTTP

Exercice

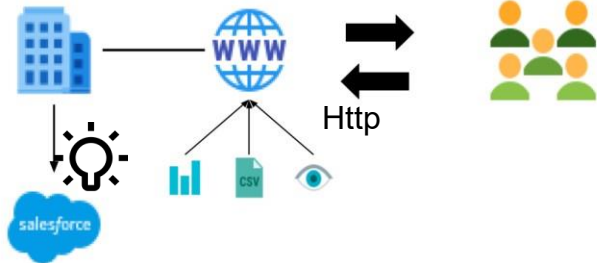
Application Programming Interface (API)



Historique des APIs

Comment les entreprises et les start-ups offraient des services sur le web à la fin des années 1990 et au début des années 2000.

Industrie



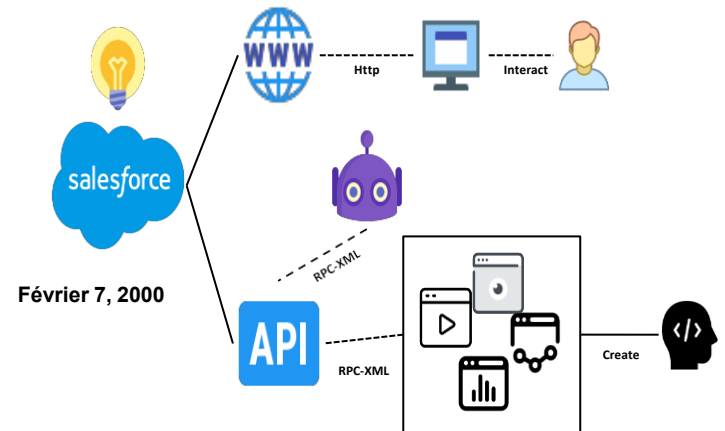
Théorie



2000



Styles architecturaux et conception d'architectures logicielles basées sur le réseau



ebay

Novembre 20, 2000

a

Juillet 16, 2002

social media



2003
delicious
api

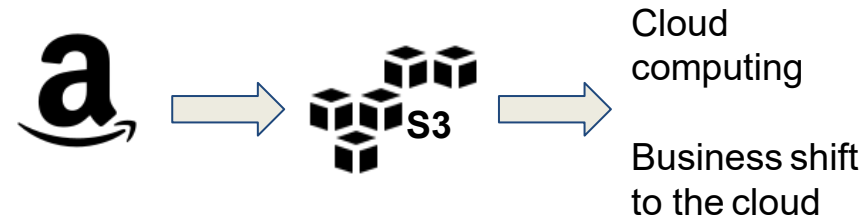
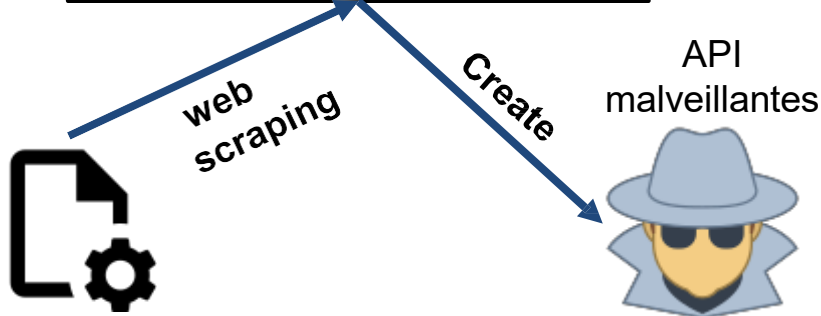
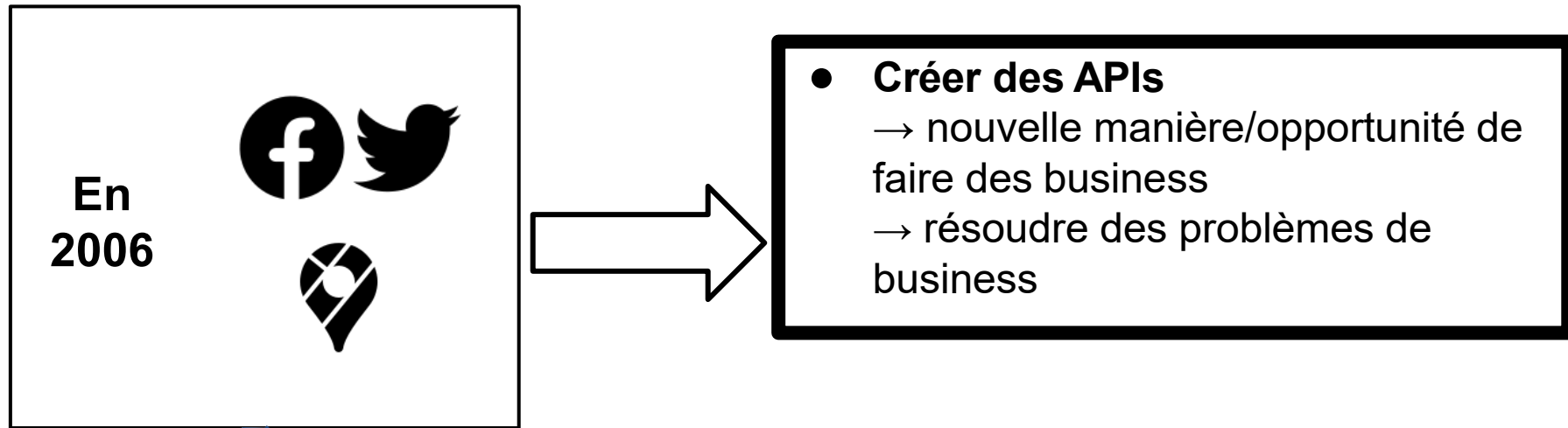


February 2004
flicker
api

2006

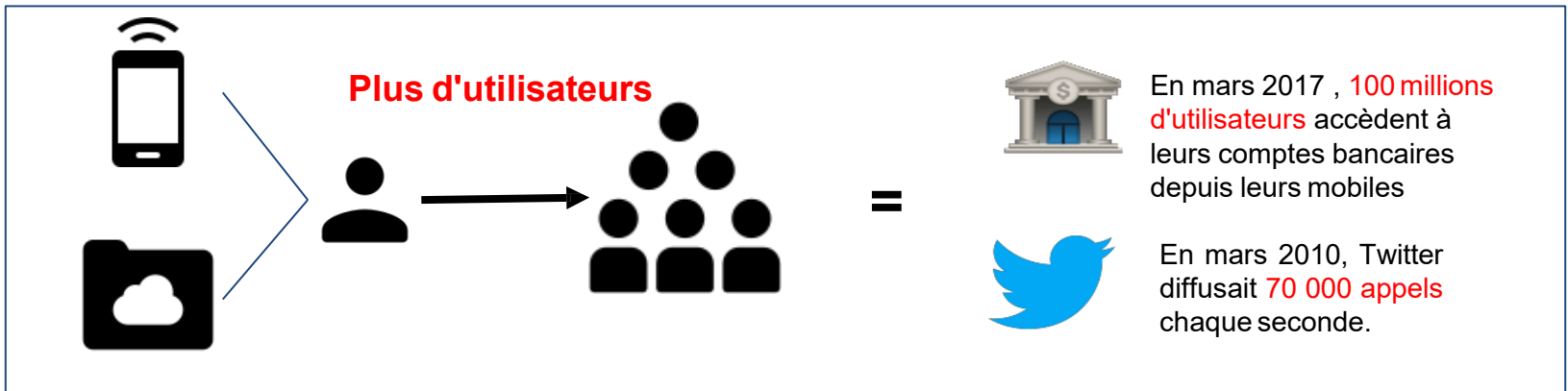


Historique des APIs



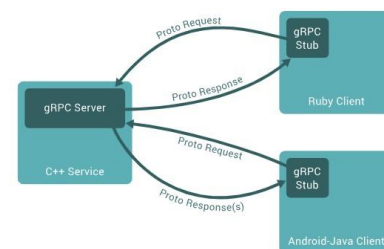
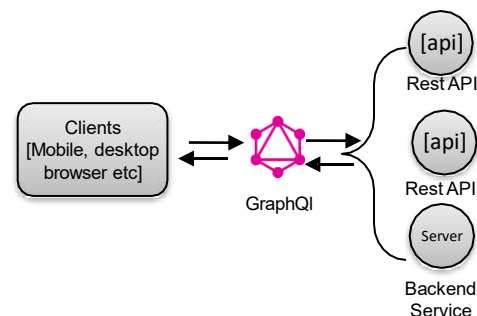
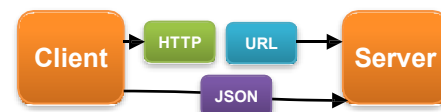
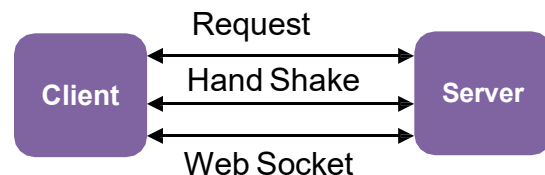
- Besoin de l'apprentissage automatique et l'analyse des sentiments
 - Utilisateur **n'était pas légal**
 - Causer des problèmes à la fois aux développeurs / et aux entreprises

Historique des APIs



Technologies utilisés pour créer l'API Web

- **RPC**
 - Ce terme a été utilisé pour la première fois en 1981
- **XML-RPC**
 - impliqué dans SOAP
- **REST**
 - Style architectural pour la conception d'applications réseau (2000)
 - Utilisation des méthodes HTTP
- **GraphQL**
 - GraphQL est un langage de requête pour les API
 - 2012 (2015 open source)
- **gRPC**
 - 2015 en tant que cadre open source pour l'utilisation de RPC à l'aide de tampons de protocole
 - Est à haute performance (grâce à HTTP/2.0),
- **Autres Techno.:** MQTT, EDI, EDA, SSE, AMQP, Webhools



Exemple d'API Android

- **API de Géolocalisation :**

- Utilisation de l'API **Google Map** dans l'application Android Studio
 - Le service gratuit est limité (25 000 chargements de cartes par jour)
 - Les entreprises utilisent Google Map: **Uber, Lyft, YASSIR**, etc.

- **API Météo** dans l'application Android Studio

- Une API météo Android est un service Web RESTful..
- **ex.** Dark Sky, Weatherbit, OpenWeatherMap, National Weather Service, Weather2020

➤ **Intégration API** → Utilisabilité

- API UX design **[*]**

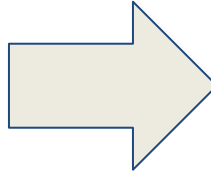
Exercice

Intégrez **une API météo** dans une application Android Studio. Dans ce scénario, en utilisant la bibliothèque **Retrofit**

- Ajouter la dépendance pour la bibliothèque Retrofit
- Créez une interface API Météo
- Créer une classe de modèle pour représenter les données renvoyées par l'API
- Dans votre activité ou fragment, effectuez l'appel API

Cycle de vie des API

API en tant
que produit



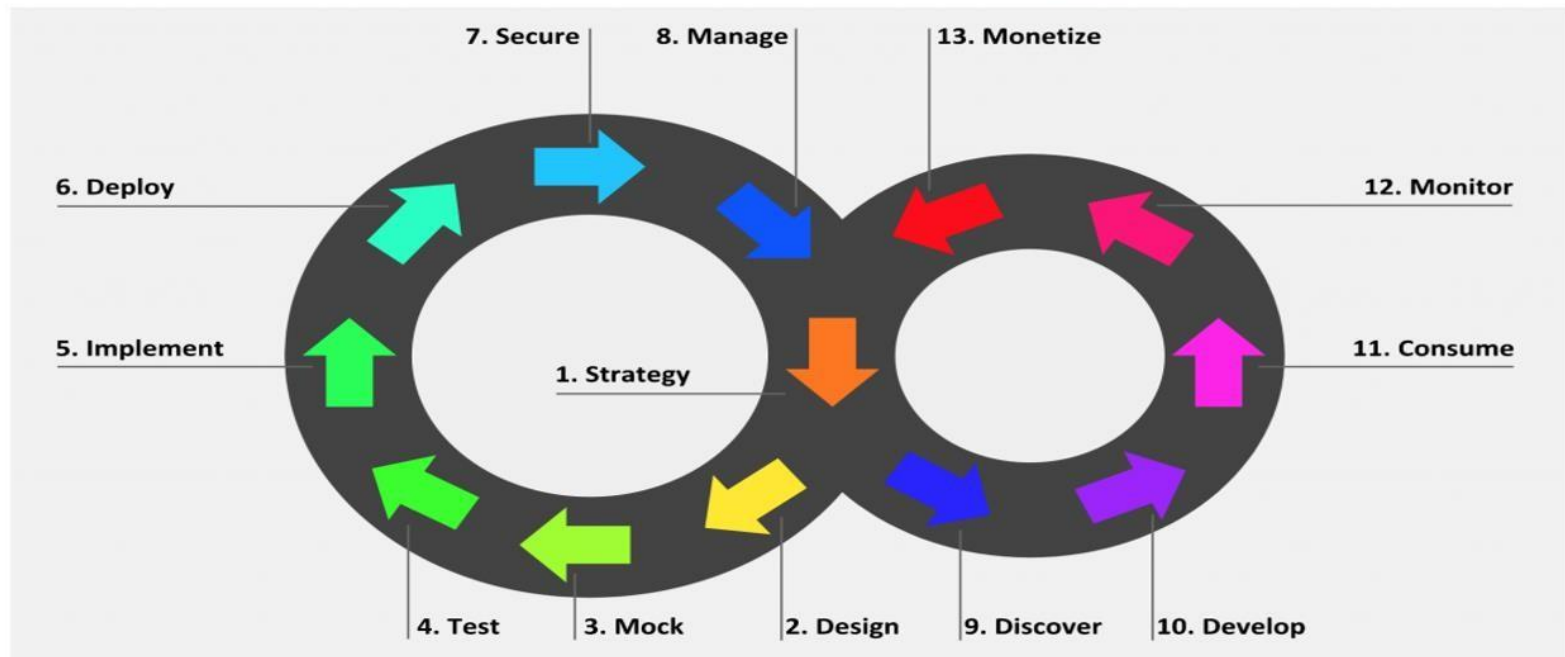
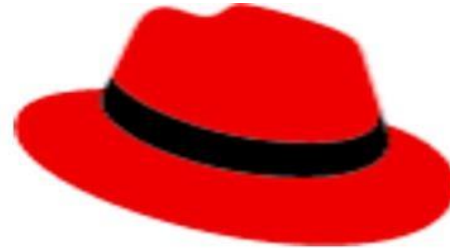
Mais chacun a sa propre vision
du **cycle de vie des API** [*]

- Axway
- Redhat
- Software AG
- IBM
- WSO2
- Sensedia
- MuleSoft
- Kong
- Microsoft Azure

Cycle de vie des API

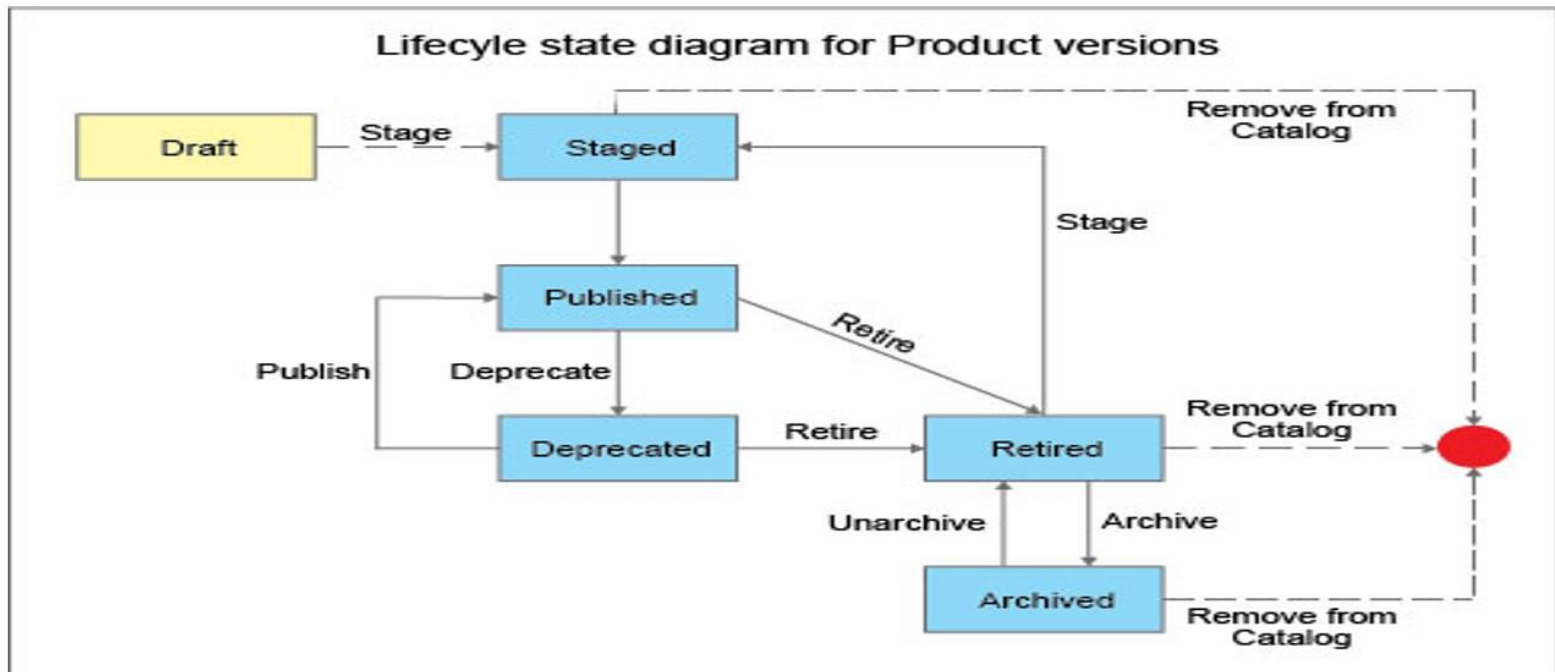
Comment **Redhat** voit le cycle de vie de l'API ?

Redhat



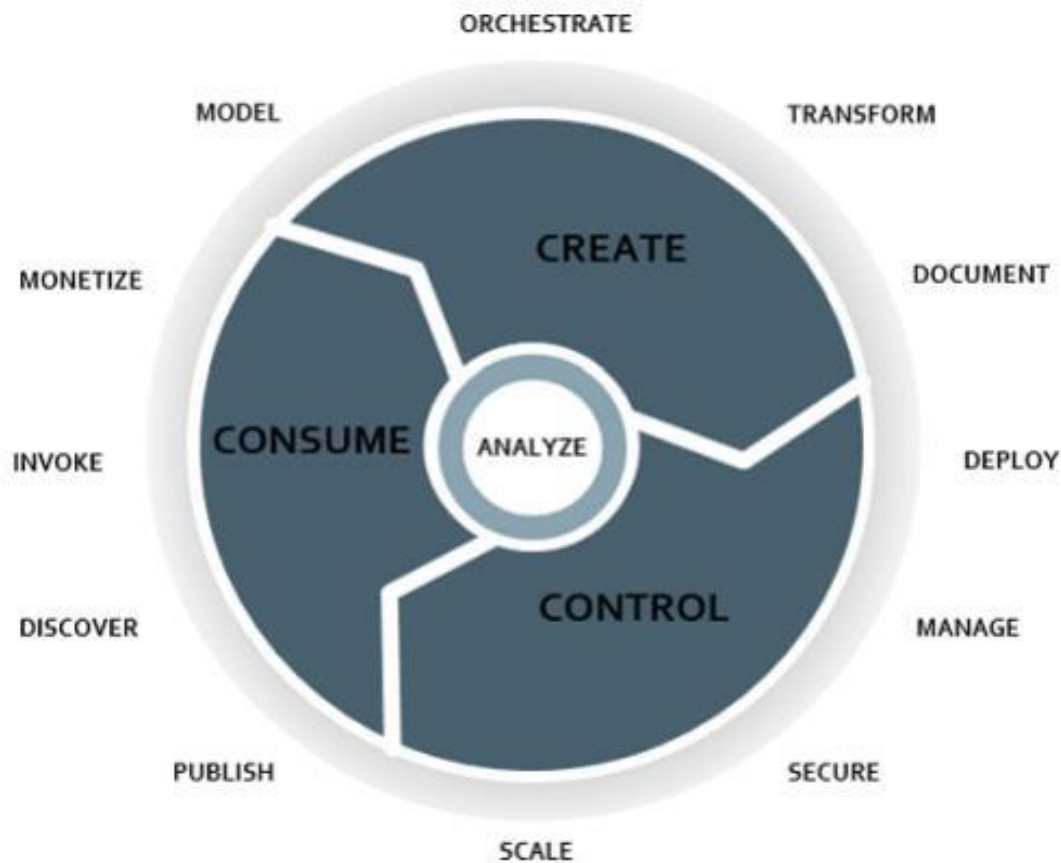
Cycle de vie des API

Comment **IBM** voit le cycle de vie des API ?

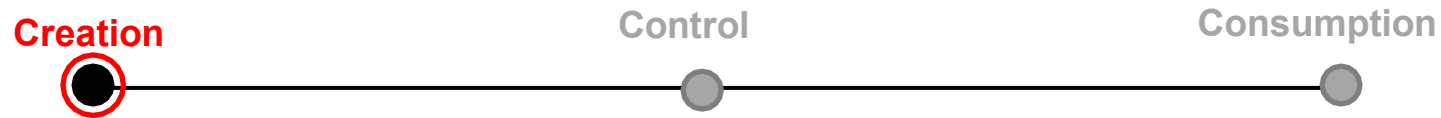


Cycle de vie des API

comment **Axway** perçoit le cycle de vie des API ?



Cycle de vie des API



Création: La partie Création contient des processus comme : Modèle (visuel/programmation) : spécifiez les données requises pour les points de terminaison API (API endpoints).

- **Orchestrer** : unifier le format de données, que nous obtenons de nombreuses sources.
- **Transformer** : convertir le format existant tel que XML en format standard tels que JSON
- **Document** : auto génère automatiquement un document et un code exemplaires pour les modèles et les activités API.

Cycle de vie des API



Contrôle: L'activité Contrôle composée de :

☐ Déployer

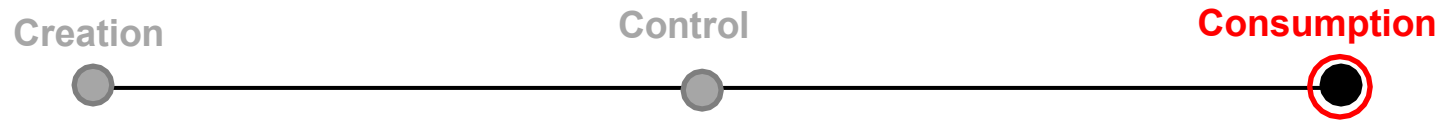
- Déploiement instantané sans délai entre la livraison du code et le déploiement
 - nous devons donc utiliser des outils DevOps (CI/CD),
- pour les petits projets, vous devez proposer une solution sur laquelle vous pouvez déployer en un seul clic.

- ## ☐ Ajoutez une **gestion** pour contrôler l'accès à notre API via une limitation de débit,
- lorsque vous avez beaucoup d'API vous ne pouvez pas les gérer vous aurez besoin d'un catalogue d'API pour gérer: publié, obsolète, retiré, ainsi que la gestion des versions et **la limite de débit** pour protéger le système contre **une utilisation élevée du trafic**.

- ## ☐ **API sécurité:** offrir une sécurité API

- ## ☐ Augmentez (**Scale**) ou réduisez l'infrastructure pour exécuter les applications côté serveur et s'ajuste automatiquement au trafic sans interaction manuelle.

Cycle de vie des API



Consommation: Enfin les composantes de la Consommation qui sont :

- ❑ **publier** l'API auprès de groupes internes, de partenaires ou du public via un catalogue central vers un portail de développeurs d'API pour être facilement consommé par eux
- ❑ **le marketing** ne doit pas être négligé. Par exemple, les API ouvertes ou publiques nécessitent une image de marque attrayante et sont également promues via les différents canaux de marketing.
- ❑ **invoquer**, exécuter des opérations d'API, peut générer des SDK à partir du portail des développeurs d'API dans leur langage préféré.
- ❑ **monétisez** l'utilisation du suivi et appliquez une politique de plan tarifaire pour générer des revenus API.

Partie Industrielle : Sites Web et Outils

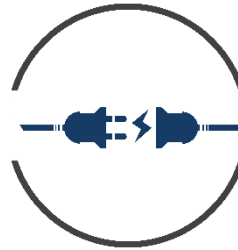
Microcks



- Un outil de mockup et de test d'API qui aide lors des phases de prototypage.

<https://microcks.io/>

FireApis



- Plateforme de création d'API sans code
- Aucun souci d'installation, d'évolutivité, de disponibilité.

<https://fireapis.com/>

Apicurio



- **développer en collaboration de meilleurs contrats API.**
- **architectes, propriétaires de produits, concepteurs et développeurs.**

<https://www.apicur.io/>

Comprendre les services Web RESTful

- Fondamentaux HTTP
- Services Web RESTful



Agenda

Introduction

API

Fondamentaux
HTTP

API REST

Bibliothèques
HTTP

Exercice

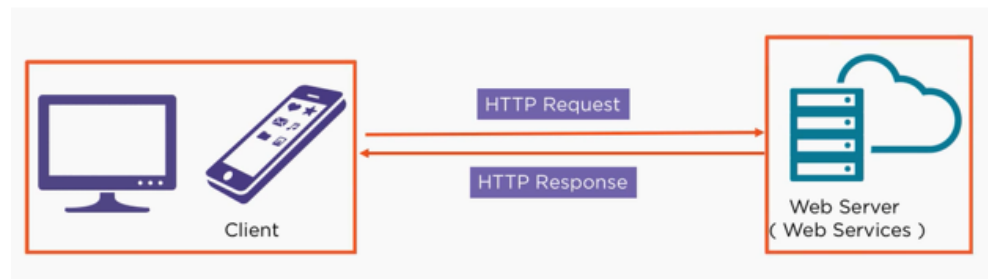
Fondamentaux HTTP

- Requête HTTP
- Réponse HTTP
- Code d'état
- Comment se déroule la communication entre le client et le serveur



HTTP Hyper Text Trasfert Protocol

- Le protocole HTTP (HyperText Transfer Protocol: Il s'agit d'un protocole de niveau application pour les systèmes d'information distribués, collaboratifs et hypermédia.
- **La version 1.0** du protocole (la plus utilisée) : transférer des messages avec des en-têtes décrivant le contenu du message en utilisant un codage de type MIME
- **Le but** du HTTP : transfert de fichiers (essentiellement au format HTML) localisé grâce à une URL entre un navigateur (le client) et un serveur Web .



HTTP Hyper text Trasfert Protocol

Fonctionnement (très simple en HTTP/1.0)

- **connexion**
 - **demande** (GET) d'un document
 - **renvoi** du document (status=200) ou d'une erreur
 - **déconnexion**
-
- Le dialogue plus complexe en cas d'authentification
 - **Optimisation** : une série de plusieurs requêtes sur une connexion

Exemple

- On souhaite obtenir la page :

<http://www.mail.dz/exemple/linux.html>

- C'est donc cette adresse qui est tapée dans la barre d'adresse du navigateur
- La code HTML de la page est :

```
<html>
<body>
<p> <h1> Ceci est une image , c'est le Logo de LINUX </h1><br>
 </p>
</body>
<html>
```

Exemple

❑ Le navigateur envoie

GET /exemple/linux.html HTTP/1.1
Connection: Keep-Alive
User-Agent: Mozilla/5.0 (compatible; Konqueror/3.1; Linux; fr)
Referer: <http://www.mail.dz/exemple/>
Pragma: no-cache
Cache-control: no-cache
Accept: text/html, image/jpeg, image/png, text/*, image/*, */*
Accept-Encoding: x-gzip, x-deflate, gzip, deflate, identity
Accept-Charset: iso-8859-1, utf-8;q=0.5, */q=0.5
Accept-Language: fr,en
Host: www.mail.dz

❑ Le serveur répond et envoie la page web

HTTP/1.1 200 OK

Date: Tue, 22 Jun 2004 13:18:15 GMT
Server: Apache/1.3.26 (Unix) Debian GNU/Linux PHP/4.1.2 mod_ssl/2.8.9 OpenSSL/0.9.6g
DAV/1.0.3
Last-Modified: Tue, 22 Jun 2004 13:15:43 GMT
ETag: "63f3d-8e-40d830ff"
Accept-Ranges: bytes
Content-Length: 142
Keep-Alive: timeout=15, max=2000
Connection: Keep-Alive
Content-Type: text/html

Exemple

Le navigateur lit que le fichier linux.gif quidoit être affiché , il demande de récupérer l'image

GET /exemple/linux.gif HTTP/1.1

Host: www.mail.dz

User-Agent: Mozilla/5.0 (X11; U; Linux i686; fr; rv:1.8.1.8) Gecko/20071030
Fedora/2.0.0.8-2.fc8 Firefox/2.0.0.8

Accept: image/png,*/*;q=0.5

Accept-Language: fr,fr-fr;q=0.8,en-us;q=0.5,en;q=0.3

Accept-Encoding: gzip,deflate

Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7

Keep-Alive: 300

Connection: keep-alive

Referer: <http://www.mail.dz/exemple/linux.html>

Exemple

Le serveur envoi l'image et le navigateur l'affiche :

HTTP/1.1 200 OK

Date: Sat, 24 Apr 2010 19:36:24 GMT

Server: Apache/2.2.6 (Fedora)

Last-Modified: Sat, 24 Apr 2010 11:27:31G

ETag: "3a7145-19ae-d4566ec0"

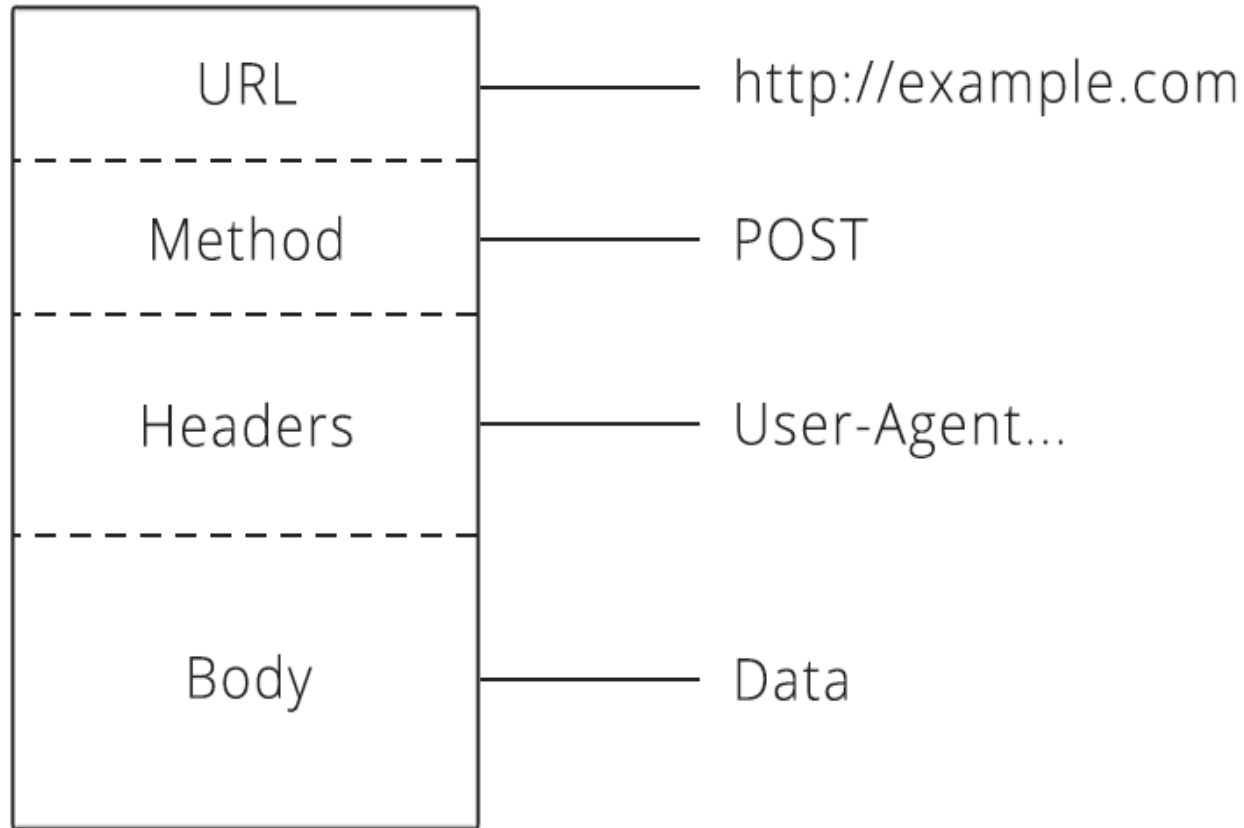
Accept-Ranges: bytes

Content-Length: 6574

Connection: close

Content-Type: image/gif

Format de la requête



Request

Format de la requête

- Envoyé par le client au serveur

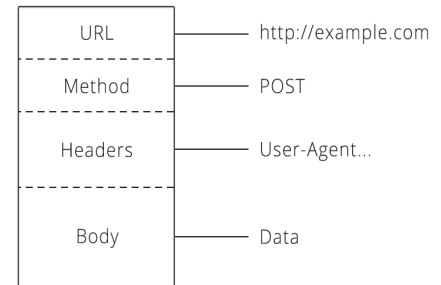
<Méthode> <URI> HTTP/<Version>

[<Champ d'entête>: <Valeur>]

[<tab><Suite Valeur si >1024>]

ligne blanche

[corps de la requête pour la méthode POST]



Request

GET /docu2.html HTTP/1.0

Accept: www/source

Accept: text/html

Accept: image/gif

User-Agent: Lynx/2.2 libwww/2.14

From: alice@pays.merveilles.net

** une ligne blanche **

POST /script HTTP/1.0

Accept: www/source

Accept: text/html

Accept: image/gif

User-Agent: Lynx/2.2 libwww/2.14

From: alice@pays.merveilles.net

Content-Length: 24

** une ligne blanche **

name1=value1&

name2=value2

Format de la requête

- Une requête HTTP est un ensemble de lignes envoyé au serveur par le navigateur.
- Elle comprend:
 - **une ligne de requête:**
 - La ligne comprend trois éléments devant être séparé par un espace:
 - La méthode,
 - L'URL,
 - La version du protocole utilisé par le client (généralement *HTTP/1.0*)
 - **Les champs d'en-tête de la requête:**
 - il s'agit d'un ensemble de lignes facultatives permettant de donner des informations supplémentaires sur la requête et/ou le client (Navigateur, système d'exploitation,...).
 - Chacune de ces lignes est composé d'un nom qualifiant le type d'en-tête, suivi de deux points (:) et de la valeur de l'en-tête
 - **Le corps de la requête:** C'est un ensemble de lignes optionnels et permettant par exemple un envoi de données par une commande POST lors de l'envoi de données au serveur par un formulaire

Format de la requête

- Demander une ressource
 - **GET** : récupérer une ressource
 - **POST**: envoyer des données et récupérer une ressource
- Avoir de l'information sur une ressource
 - **HEAD** : connaître ses caractéristiques
 - **OPTIONS** : connaître les options qui lui sont applicables
- Mettre à jour une ressource à distance
 - **PUT** : la créer ou remplacer son contenu
 - **DELETE** : la détruire

Format de la requête

- **OPTIONS**
demande des options de communication disponibles
- **TRACE**
retourne le corps de la requête intacte (débugage)
- **CONNECT**
- + Nouvelles extensions de HTTP/U et HTTP/MU
NOTIFY, ... (UPnP)

Format d'une Réponse HTTP

Réponse envoyé par le serveur au client

HTTP/<Version> <Status> <Commentaire Status>

Content-Type: <Type MIME du contenu>

[< Champ d 'entête >: <Valeur>]

[<tab><Suite Valeur si >1024>]

Ligne blanche

Document

HTTP/1.0 200 OK

Date: Wed, 02Feb97 23:04:12 GMT

Server: NCSA/1.1

MIME-version: 1.0

Last-modified: Mon, 15Nov96 23:33:16 GMT

Content-type: text/html

Content-length: 2345

* une ligne blanche *

<HTML><HEAD><TITLE> ...

</BODY></HTML>

Code de réponse HTTP

- Ce sont les codes que vous voyez lorsque le navigateur n'arrive pas à vous fournir la page demandée.
- Le code de réponse est constitué de trois chiffres:
 - Le premier indique la classe de statut:
 - **1xx : information** ,
 - **2xx : succès** ,
 - **3xx : redirection** ,
 - **4xx : erreur du client** ,
 - **5xx : erreur du serveur**
 - les autres indiquent la nature exacte d'erreur.

Code de réponse HTTP

1XX Informational	
100	Continue
101	Switching Protocols
102	Processing

2XX Success	
200	OK
201	Created
202	Accepted
203	Non-authoritative Information
204	No Content
205	Reset Content
206	Partial Content
207	Multi-Status
208	Already Reported
226	IM Used

3XX Redirectional	
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
305	Use Proxy
307	Temporary Redirect
308	Permanent Redirect

4XX Client Error	
400	Bad Request
401	Unauthorized
402	Payment Required
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required
408	Request Timeout

4XX Client Error Continued	
409	Conflict
410	Gone
411	Length Required
412	Precondition Failed
413	Payload Too Large
414	Request-URI Too Long
415	Unsupported Media Type
416	Requested Range Not Satisfiable
417	Expectation Failed
418	I'm a teapot
421	Misdirected Request
422	Unprocessable Entity
423	Locked
424	Failed Dependency
426	Upgrade Required
428	Precondition Required
429	Too Many Requests
431	Request Header Fields Too Large
444	Connection Closed Without Response
451	Unavailable For Legal Reasons
499	Client Closed Request

5XX Server Error	
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported
506	Variant Also Negotiates
507	Insufficient Storage
508	Loop Detected
510	Not Extended
511	Network Authentication Required
599	Network Connect Timeout Error

Services Web REST

- Concepts de base
- Configuration du serveur
- Client REST



Agenda

Introduction

API

Fondamentaux
HTTP

API REST

Bibliothèques
HTTP

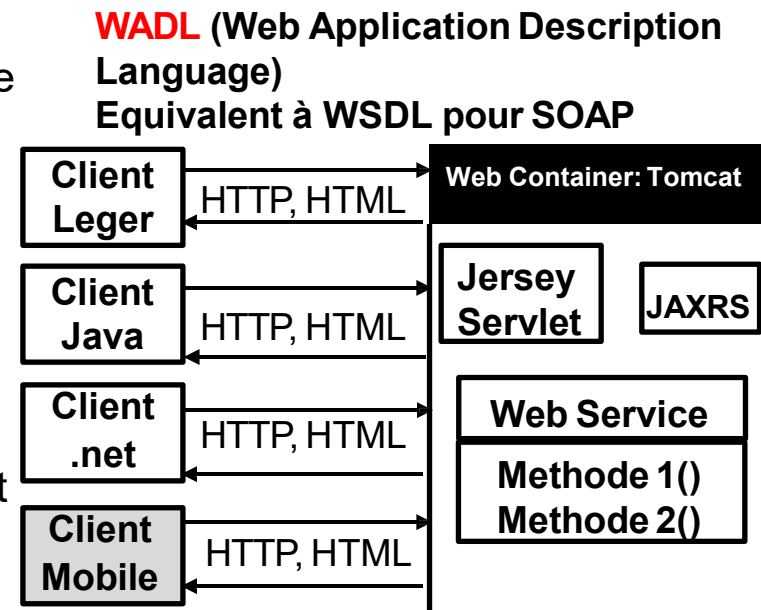
Exercice

Web service

- Deux catégories de web service:
 - ❑ Web service basé sur SOAP (1999)
 - Concepts fondamentaux: SOAP, WSDL, UDDI
 - Pb pour consommer un web service SOAP :
 - Utilise *forcément* XML (XML est **verbeux** ▪)
 - Charger un doc XML à partir de flux (parceur !!) en mémoire sous forme d'un arbre
 - Transport et chargement en mémoire -> temps !!
 - Web service basé sur REST (2000)
 - RestFul est une norme
 - Style d'architecture pour créer un système distribué pour échanger les données
 - REST Utilise HTTP (SOAP n'utilise pas forcément HTTP)

RESTful

- REST (Representational State Transfert) ou RESTful est un style d'architecture pour les systèmes hypermédia distribués,
- Créé par *Roy Fielding* en **2000** dans le chapitre 5 de sa thèse de doctorat.
- REST est un style d'architecture permettant de construire des applications (*Web, Intranet, Web Service*)
- Il s'agit d'un ensemble de conventions et de bonnes pratiques à respecter et non d'une technologie à part entière.
- L'architecture REST utilise les spécifications originelles du protocole **HTTP**, plutôt que de réinventer une surcouche (comme le font SOAP ou XML-RPC par exemple)



Les 5 règles de RESTful

- **Règle n° 1:**
- l'URI comme identifiant des ressources:
 - Respecter un standard pour construire les URLs
- **Règle n° 2:**
- Les méthodes HTTP comme identifiant des opérations:
 - **POST:** Pour ajouter (Create)
 - **GET:** Pour consulter (Read)
 - **PUT:** pour mise à jour (Update) ou PATCH pour update partiel
 - **DELETE:** pour supprimer (DELETE)
- **Règle n° 3:**
- Les réponses HTTP comme représentation de ressources:
 - Réponse HTTP en différents format (XML, JSON,) en fonction de la demande du client
- **Règle n° 4:**
- Les liens comme relation entre ressources:
 - `<a href « -- » ref=« payment"> Effectuer un paiement </a>`
- **Règle n° 5:**
- Un paramètre comme jeton d'authentification:
 - Envoyer un jeton d'authentification différent pour chaque requête

Les 5 règles de RESTful

- Règle n° 1:
- l'URI comme identifiant des ressources:
 - Respecter un standard pour construire les URLs
- REST se base sur les **URI (Uniform Resource identifier)** afin d'identifier une ressource.
- Ainsi une application se doit de construire ces URI (et donc ses URL) de manière précise, en tenant compte des contraintes REST.
- Il est nécessaire de prendre en compte la hiérarchie des ressources et la sémantique des URL pour les éditer:
- Liste des livres
 - **GET** <http://mywebsite.com/books>
- Filtre et tri sur les livres
 - **GET** <http://mywebsite.com/books?filtre=policier&tri=asc>
- Consulter un livre
 - **GET** <http://mywebsite.com/books/87>
- Tous les commentaires sur un livre
 - **GET** <http://mywebsite.com/books/comments>
- Ajouter un livre
 - **GET** <http://mywebsite.com/books/comments/87/comments>
- Mettre à jour un livre
 - **PUT** <http://mywebsite.com/books/comments/5>
- Supprimer un livre
 - **DELETE** <http://mywebsite.com/books/comments/5>

Les 5 règles de RESTful

- Règle n° 2:
 - Les méthodes HTTP comme identifiant des opérations:
-
- La seconde règle d'une architecture REST est d'utiliser les verbes HTTP existants plutôt que d'inclure l'opération dans l'URI de la ressource.
 - Ainsi, généralement pour une ressource, il y a 4 opérations possibles (CRUD):
 - Créer (create)
 - Afficher (read)
 - Mettre à jour (update)
 - Supprimer (delete)
 - HTTP propose les verbes correspondant:
 - Créer (create) → **POST**
 - Afficher (read) → **GET**
 - Mettre à jour (update) → **PUT ou PATCH**
 - Supprimer (delete) → **DELETE**

Les 5 règles de RESTful

- Règle n° 3:
- Les réponses HTTP comme représentation de ressources:
 - Réponse HTTP en différents formats (XML, JSON,) en fonction de la demande du client:
- Il est important d'avoir à l'esprit que la réponse envoyée n'est pas une ressource, mais plutôt la représentation d'une ressource
- Ainsi, une ressource peut avoir plusieurs représentations dans des formats divers: **HTML, XML, CSV, JSON, etc.**
- C'est au client de définir quel format de réponse il souhaite recevoir via l'entête http **Accept**
- Il est possible de définir plusieurs formats
 - Demander une réponse en HTML
 - GET /books
 - Host mywebsite.com
 - **Accept: application/xml**
 - Demander une réponse en XML
 - GET /books
 - Host mywebsite.com
 - **Accept: application/json**

Les 5 règles de RESTful

- Règle n° 4:
- Les liens comme relation entre ressources:
 - `<a href « -- » ref=« payment »> Effectuer un paiement </a>`
- Les **liens** d'une ressource vers une autre indique la présence d'une **relation**.
- Il est cependant possible de la décrire afin d'améliorer la compréhension du système.
- Exemple quand on consulte une ressource qui représente un produit d'une catégorie, on trouve dans la description du produit un lien significatif qui permet de consulter la catégorie de ce produit
- Ainsi l'IANA donne une liste de relation parmi lesquelles:
 - Content, edit, next, last , payment, etc.
- La liste complète sur le site de l'IANA:
 - <https://www.iana.org/assignments/link-relations/link-relations.xhtml>

Les 5 règles de RESTful

- Règle n° 5:
 - Un paramètre comme jeton d'authentification:
 - Envoyer un jeton d'authentification différent pour chaque requête
-
- C'est l'un des sujets les plus souvent abordés quand on parle de REST: ***comment authentifier une requête?***
 - Pour authentifier les requêtes HTTP, il faut envoyer dans la requête un jeton d'authentification de la requête
 - Le premier Jeton X est obtenu au moment de l'authentification.
 - Pour chaque requête HTTP, ce jeton X peut être mélangé avec l'URL de la requête en utilisant une opération de hachage.
 - Cette opération permet d'avoir un jeton différent pour chaque requête
 - **Exemple de Token d'authentification: JWT (Json Web Token)**

Agenda

Introduction

API

Fondamentaux
HTTP

API REST

Bibliothèques
HTTP

Exercice

Bibliothèques HTTP (Client REST)

- HttpURLConnection
- Retrofit
- Volley



BIBLIOTHÈQUES RÉSEAU POUR ANDROID

- Toutes les applications Android utilisent une **API Web REST** pour le transfert de données.
 - Auparavant, les développeurs préféraient écrire leur propre code de données pour récupérer et analyser les données.
 - Gamme de bibliothèques clientes REST :
 - Accélérez le développement en réduisant les efforts de codage et analysez efficacement les données.
 - Android **Volley** et **Retrofit** sont aujourd'hui les bibliothèques les plus utilisées pour accéder aux API Web REST..

BIBLIOTHÈQUES RÉSEAU POUR ANDROID

API	Definition	License	Important Points
Android Asynchronous HttpClient	An asynchronous callback- based Http client for Android built on top of Apache's HttpClient libraries.	Apache License 2.0	• Views that require multiple API calls. • Run extremely slow. • Take multiple seconds to load.
KSOAP2	The ksoap2-android project provides a lightweight and efficient SOAP client library for the Android platform.	MIT License	• Allows loading Data from SOAP web service in Android Application.
PICASSO	Picasso allows for image loading in the application by handling ImageView recycling, downloads cancelation, and offering memory efficient image transformations, including automatic memory and diskcaching.	Apache License 2.0	• Allows loading images from assets, resources, files, URLs and content providers. • Does not support GIFs.
OKHTTP	OkHttp is an efficient HTTP client providing features such as SPDY, transparent compression, connection pooling and response caching.	Apache License 2.0	• OkHttp is an HTTP client, which supports HTTP/2 and SPDY. • OkHttp depends on Okio.
Volley	An HTTP library that makes networking for Android apps easier and faster.	Apache License 2.0	• Volley can integrate with most popular HTTP clients such as OkHttp, etc.
Retrofit	Retrofit is a REST client that sits on top of OkHttp and makes very easy common networking tasks.	Apache License 2.0	• Retrofit is a type-safe HTTP client for Android and Java.

Caractéristiques du client REST

**Complexité du
code**

**Chargement des
images**

Performance

**Priorisation des
demandes**

Requêtes CRUD

Extensibilité

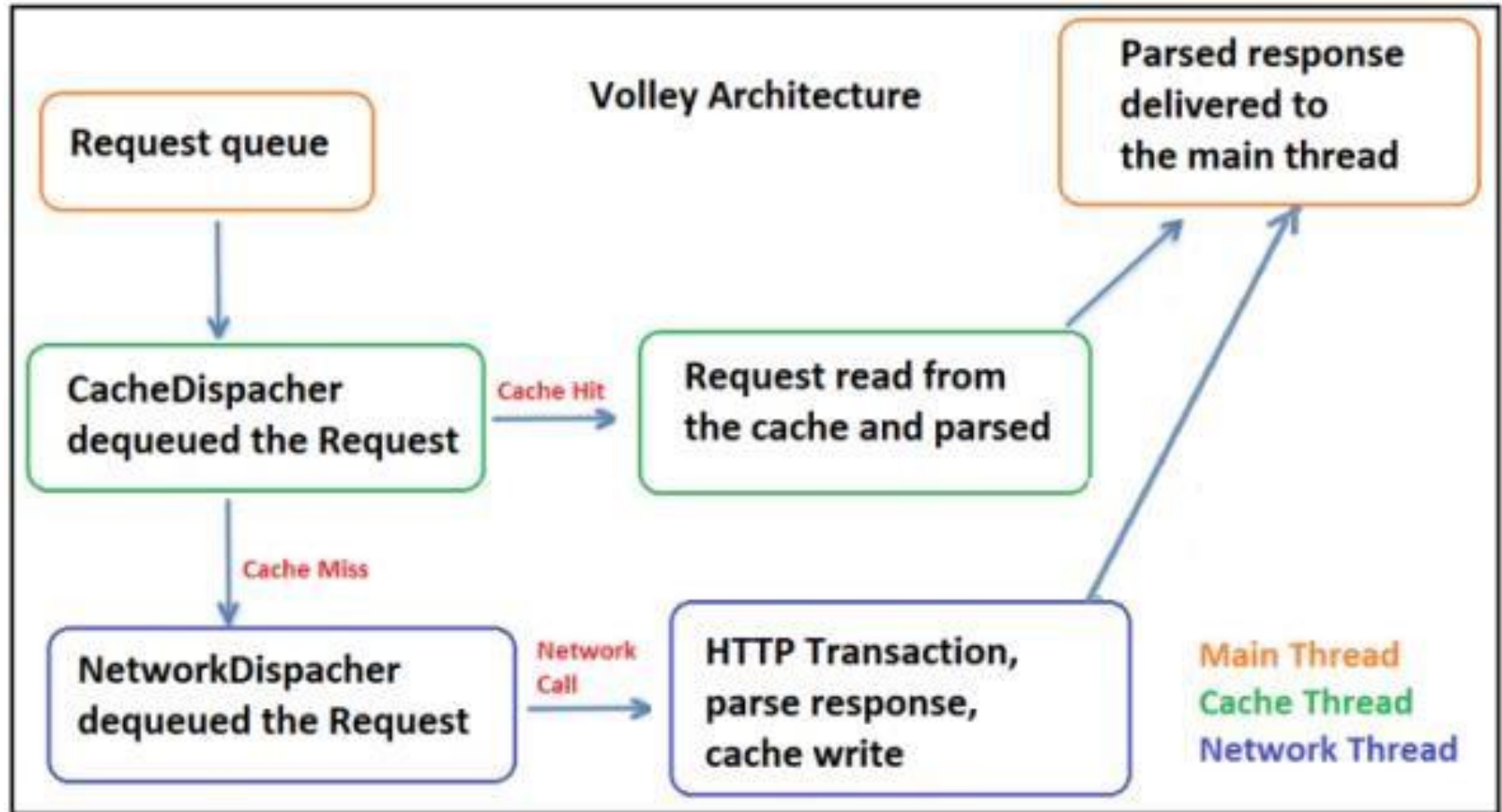
URL dynamiques

**Mise en cache
Android**

Bibliothèques HTTP populaires : client REST

- **Volley** (created by Google)
 - <https://android.googlesource.com/platform/frameworks/volley>
- **Retrofit** (Created by Square)
 - <https://github.com/square/retrofit>
- **Android Async Http** <https://github.com/loopj/android-async-http> Ion <https://github.com/koush/ion>

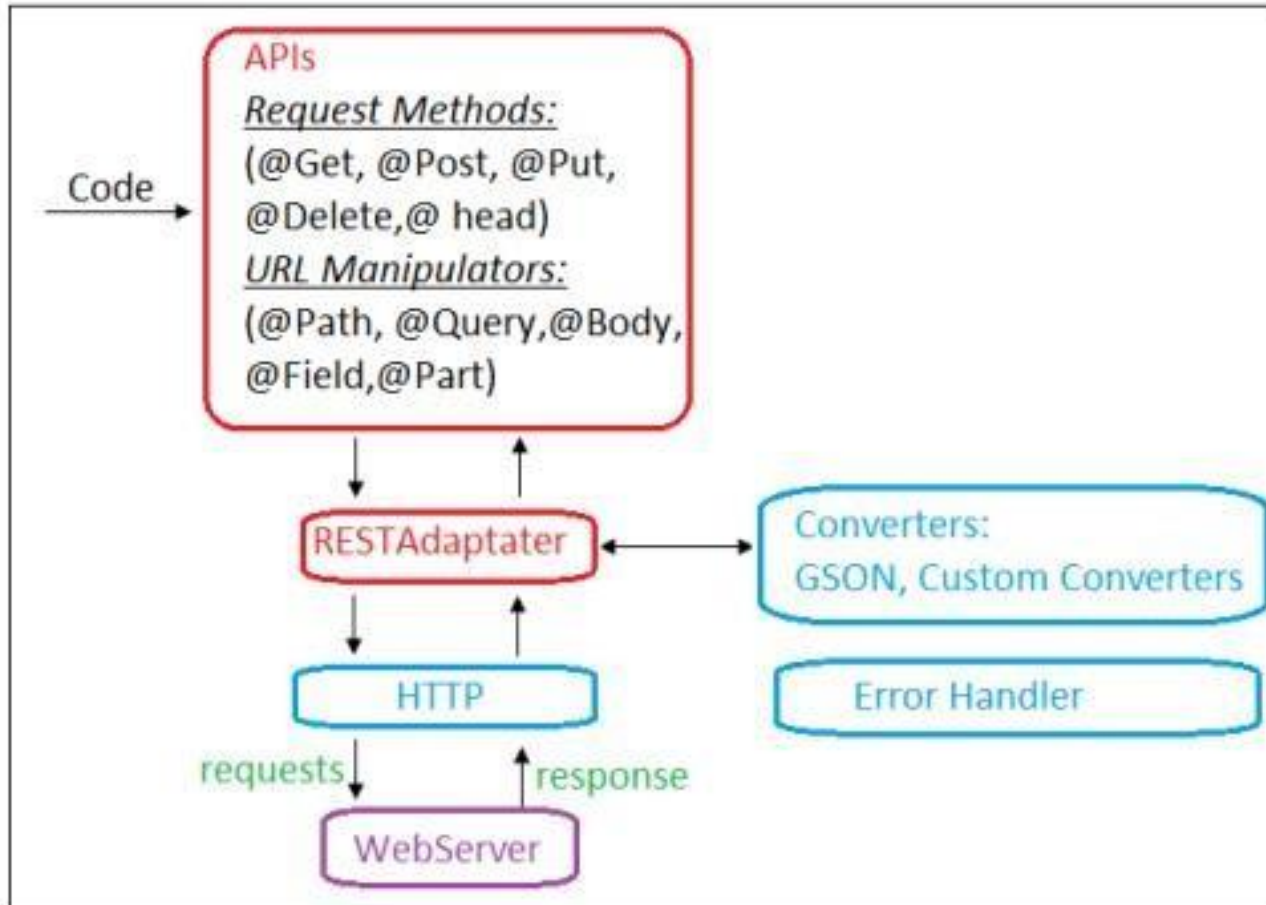
Volley: Architecture



API Retrofit

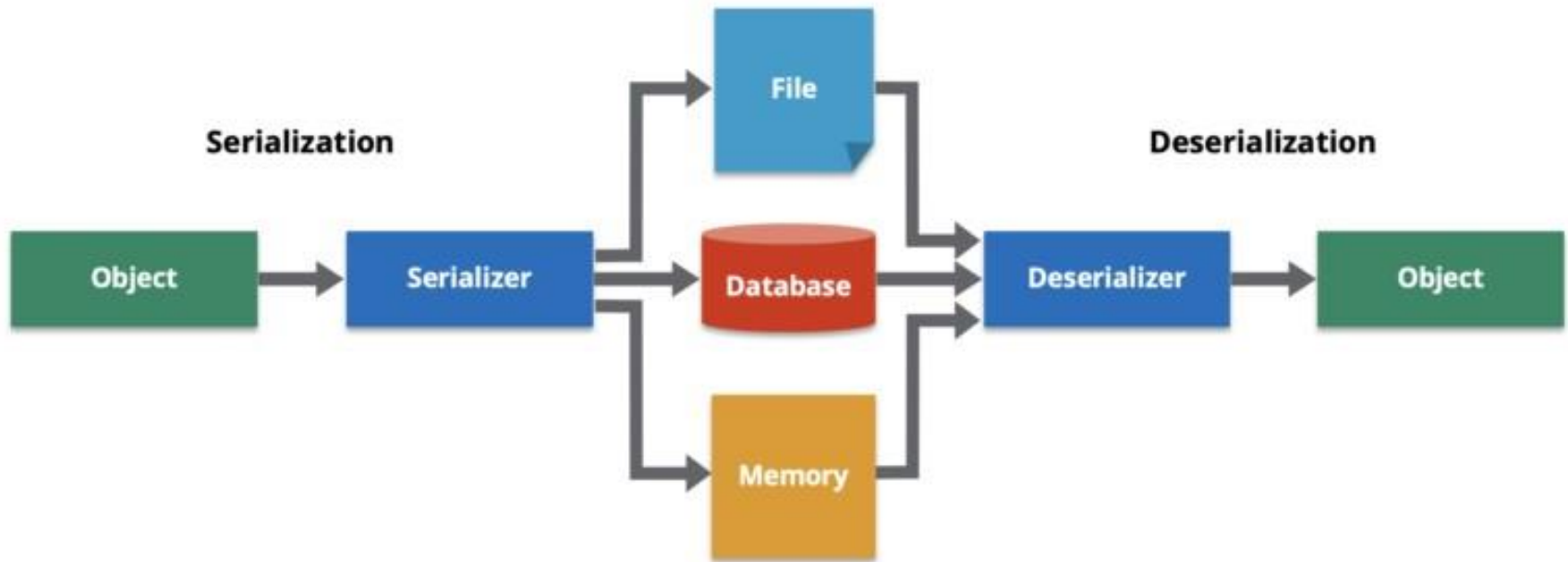
- Retrofit est une bibliothèque propre, simple et légère pour Android développée par Square.
- Permet à n'importe quelle application Android de communiquer avec des bases de données distantes
- Retrofit est un client REST qui repose sur OkHttp et effectue des tâches réseau courantes très simples.
- Retrofit peut effectuer des requêtes Async et Sync avec une analyse JSON automatique sans aucun effort.
- Licence : Licence Apache 2.0

Retrofit: Architecture



Besoin de conversion/parseur de format d'échange

Serialization/Deserialization



Convertir un objet Java/Kotlin dans sa représentation JSON et vice versa

1. Gson est une bibliothèque développée par Google pour travailler avec JSON en Java.
2. Prise en charge d'objets complexes
3. Fournit des méthodes simples pour convertir JSON en objets Java/Kotlin et vice versa.
4. Permet la personnalisation via des annotations ou des sérialiseurs/désérialiseurs personnalisés.

❑ Autres bibliothèques:

- Jackson, org.json, JSON-B (Java API for JSON Binding):

Exercise Volley vs Retrofit

Agenda

Introduction

API

HTTP
Fundamentals

RESTAPI

HTTP Libraries

Exercise

Agenda

Introduction

API

Fondamentaux
HTTP

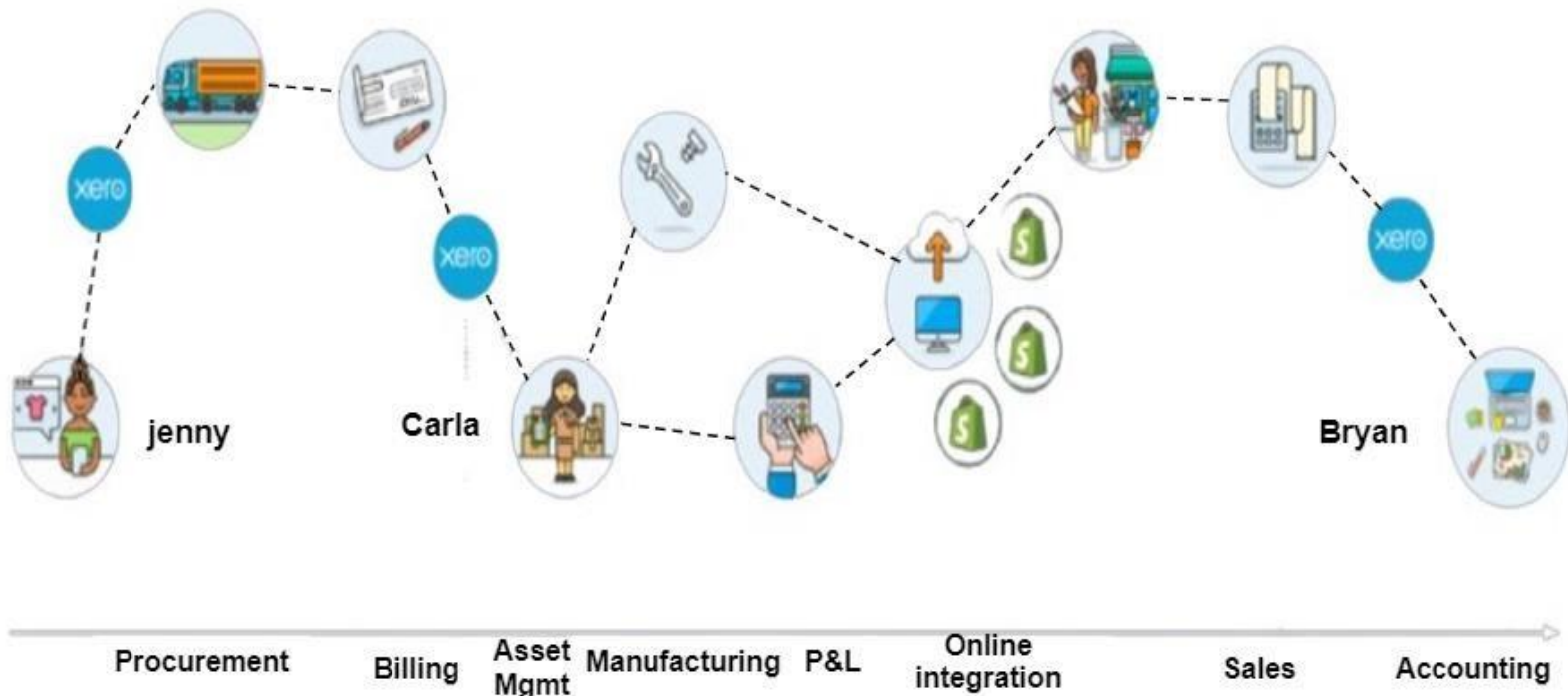
API REST

Bibliothèques
HTTP

Exercice

Exercice:

Mettre en ligne une entreprise de PME en utilisant les APIs



Questions ? !

abdelkader.ouared@univ-tiaret.dz



Tester une API

<https://api.tomorrow.io/v4/timelines>

une API qui fournit des **données météorologiques en temps réel et des prévisions** (*température, humidité, vitesseDuVent, directionDuVent, probabilitéPrécipitations*)



POSTMAN

*un **endpoint** désigne une URL spécifique*

Un endpoint est le **point d'entrée** ou l'adresse à laquelle un client (comme une application, un navigateur ou un autre service) peut envoyer des requêtes pour interagir avec une API

Comment fonctionne un endpoint ?

Un endpoint est généralement associé à une méthode HTTP spécifique (**GET**, **POST**, **PUT**, **DELETE**, etc.) et permet d'effectuer des actions comme :

- Récupérer des données.
- Créer de nouvelles données.
- Mettre à jour des données existantes.
- Supprimer des données.

Pour intégrer une API

1. Comprenez l'API et ses fonctionnalités.
2. Configurez l'environnement avec les outils nécessaires.
3. Implémentez les appels en respectant la documentation.
4. Testez et gérez les erreurs.
5. Automatisez si nécessaire.

1. Comprendre l'API (1/8)

- **Lire la documentation** : La documentation fournit les informations sur les endpoints disponibles, les méthodes HTTP (GET, POST, PUT, DELETE, etc.), les paramètres requis, les formats de réponse, et les éventuelles authentifications.
- **Formats de données** : Vérifiez si l'API utilise JSON, XML, ou un autre format pour les requêtes/réponses.
- **Limites de l'API** : Consultez les restrictions, comme les quotas de requêtes ou les temps d'attente.

2. Configurer l'environnement (2/8)

- ❑ **Choisir un langage de programmation** : Par exemple, Python, JavaScript, Java, etc.
- ❑ **Installer des bibliothèques** : Selon votre langage, des bibliothèques facilitent les requêtes HTTP :
 - Python : requests, http.client.
 - JavaScript : fetch, axios.
 - Java : HttpURLConnection, OkHttp.

3. Authentification de l'API (3/8)

- La plupart des API nécessitent une forme d'authentification. Vérifiez dans la documentation le type d'authentification requis :
 - **Clé API** : **Un jeton secret** (souvent envoyé dans les en-têtes ou les paramètres de requête).
 - **OAuth** : Nécessite un processus d'autorisation **via jetons d'accès**.
 - **Authentification basique** : Utilise **un identifiant et un mot de passe** encodés en Base64.

4. Effectuer une requête API (4/8)

- Pour interagir avec l'API, vous devez envoyer une requête HTTP à un endpoint en respectant la documentation. Voici quelques exemples dans différents langages :

5. Traiter la réponse (5/8)

- Les réponses d'une API sont souvent au format JSON ou XML. Vous devez les parser pour utiliser les données dans votre application.

6. Gérer les erreurs (6/8)

Prévoyez des mécanismes pour gérer les erreurs, comme :

- Codes HTTP (400, 401, 403, 500, etc.).
- Messages d'erreur fournis par l'API.
- Temps de réponse (ajouter un timeout)

7. Automatiser ou intégrer dans un système (7/8)

- Une fois que vous savez interagir avec l'API :
- **Créer des fonctions réutilisables** : Évitez de répéter le même code pour chaque requête.
- **Automatiser** : Intégrez les appels API dans votre workflow ou votre système.

8. Tester l'API (8/8)

- Utilisez des outils pour tester vos requêtes avant de les intégrer dans le code :
- **Postman** : Interface graphique pour construire et tester des appels API.
- **cURL** : Ligne de commande pour envoyer des requêtes HTTP.

Example: Current weather of any city using OpenWeatherMap API in Python

Python



```
import requests

def get_weather(api_key, location):
    url = "https://api.tomorrow.io/v4/timelines"
    params = {
        "apikey": api_key,
        "location": ','.join(map(str, location)), # Example location format: [lat, lon]
        "fields": ["temperature", "weatherCode"],
        "units": "metric",
        "timesteps": "1h",
        "timezone": "UTC"
    }

    response = requests.get(url, params=params)
    if response.status_code == 200:
        return response.json()
    else:
        return None

# Example usage
api_key = 'YOUR_API_KEY'
location = [34.0522, -118.2437] # Example: Los Angeles coordinates
weather_data = get_weather(api_key, location)
print(weather_data)
```

Utiliser postman

- `https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API key}`